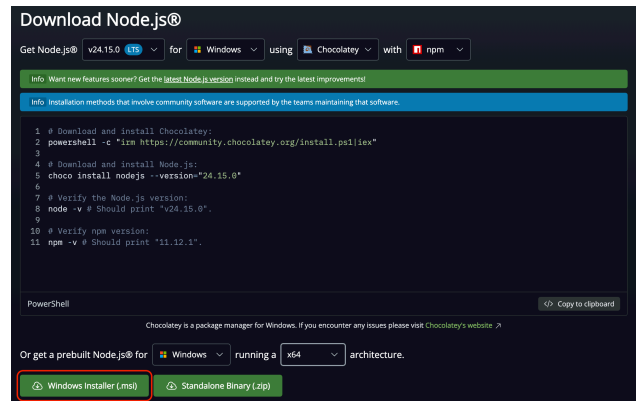


Gies P2P Chat

A completely localized, peer-to-peer web chat application designed to facilitate one-on-one communication between predefined computer pairs on a Local Area Network (LAN). Built with React, Node.js, and Socket.io, this application acts as a seamless handoff from Qualtrics surveys and supports local CSV transcript exports.

Prerequisites

- [Node.js](#) (tested in v24.15.0, LTS recommended)



Download the official macOS (.pkg) or Windows (.msi) installer directly from [Node.js](#).

- All client machines must be connected to the exact same Local Area Network (Wi-Fi or Ethernet) as the Host machine.

Local Development Setup

1. Open a Terminal and `cd` into the repository folder.
2. Install dependencies: `npm install` or `npm ci`
3. In terminal 1, run the WebSocket server: `node server.cjs`
4. In terminal 2, run the Vite development server: `npm run dev -- --host`
5. Access the development build at:
 - **Host Machine:** `http://localhost:5173/?id=DevUser1&room=1`
 - **Network Machine:** `http://[HOST_IP_ADDRESS]:5173/?id=DevUser2&room=1.`

Understanding the URL Parameters: Because this application acts as a handoff from a survey tool, the React frontend requires these variables in the URL to function:

- **id:** A unique identifier for the user. In production, this will be the auto-generated Qualtrics Response ID. For local testing, any unique string (e.g., `DevUser1`) will work.
- **room:** The pairing assignment. Users who have the same room parameter (e.g., `room=1`) will be routed into the same private WebSocket chat. A maximum of 2 users is allowed per room.

Creating Static Files for Deployment

Run `npm run build` within the `gies-p2p-chat` folder to build, or rebuild, the necessary static files for the `start-server` script.

Client Deployment (Production)

First-Time Environment Setup

⚠ This needs to be done only **once** per **host** computer!

1. Ensure `Node.js` is installed on the **Host** computer.
2. Locate and open the project folder → **gies-p2p-chat**
3. Navigate to the `Windows Scripts` or `Mac Scripts` folder (depending on the **host** computer's OS).
4. Double-click the `setup` script.
 - **Note for Mac Users:** You may need to grant the script permission to run by typing `chmod +x setup.command` in your terminal first.
5. A terminal window will open, automatically install the required dependencies, and compile the production build. Once it says "Setup Complete!" you can close the window.

Start Server

1. Navigate to the `Windows Scripts` or `Mac Scripts` folder.
2. Double-click the `start-server` script.
 - **Note for Mac Users:** You may need to grant the script permission to run by typing `chmod +x start-server.command` in your terminal first.
3. Leave this terminal window open in the background. The server is now routing messages!